Estruturação interna de aplicações monolíticas: comparação entre módulo único, arquitetura em camadas (MVC) e Clean Architecture

Victor Hugo Geroto Toledo (aluno) · Deivison Shindi Takatu (orientador) · Glauco Todesco (orientador)

Centro Universitário SENAI São Paulo — UNISENAI-SP · Campus Sorocaba · Análise e Desenvolvimento de Sistemas

Iniciação Científica · Engenharia de Software · versão preliminar do banner

1 Introdução

- **Monólito** é a forma de implantação: um único artefato com um banco de dados.
- **Camadas (MVC)** e **Clean Architecture** estruturam o código por dentro. Uma aplicação monolítica pode adotar qualquer uma delas, ou nenhuma (módulo único).
- A literatura compara monólito × microsserviços. Nas fontes revisadas, não há comparação experimental dessas estruturações internas.

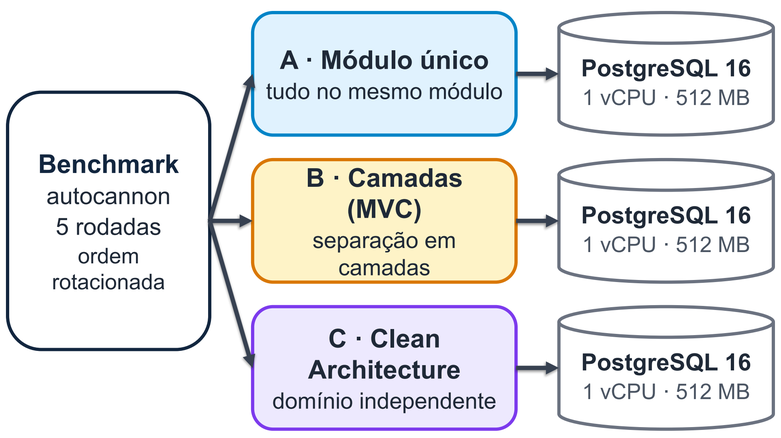
2 Problema e objetivo

- **Pergunta:** a estruturação interna de uma mesma aplicação monolítica altera desempenho, escalabilidade e manutenibilidade?
- **Objetivo:** medir as três estruturações em condições idênticas.

3 Metodologia

- Mesma API de tarefas, implementada 3 vezes: **A** módulo único · **B** camadas (MVC) · **C** Clean Architecture.
- **Constantes:** contrato HTTP, TypeScript/Node 22, PostgreSQL 16, Docker, 1 vCPU e 512 MB.
- **autocannon:** 4 cenários (10 conexões), 1 a 100 conexões e 0,5 a 2 vCPU.
- **5 rodadas** por condição, ordem rotacionada; Kruskal-Wallis com correção de Bonferroni.

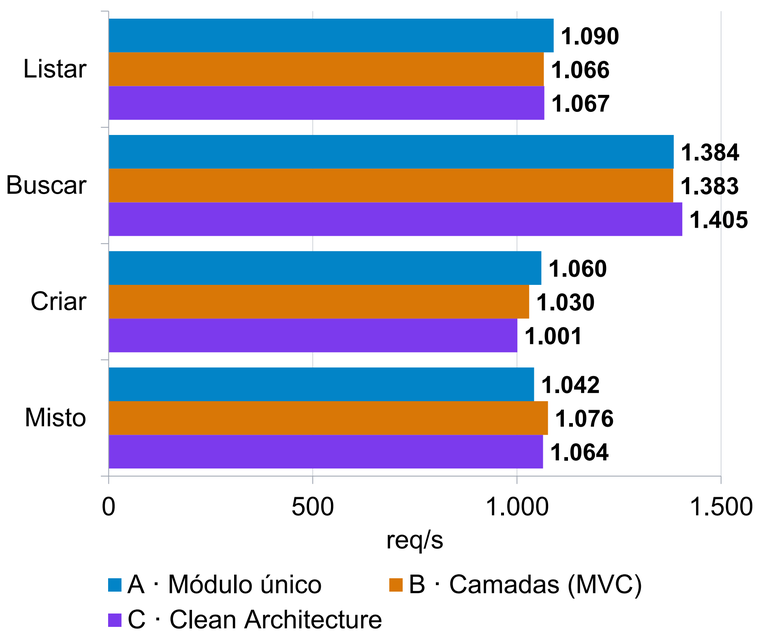
Figura 1 – Ambiente experimental (um hospedeiro, Docker, mesmos limites por serviço)



Fonte: elaborado pelos autores (2026).

4 Resultados

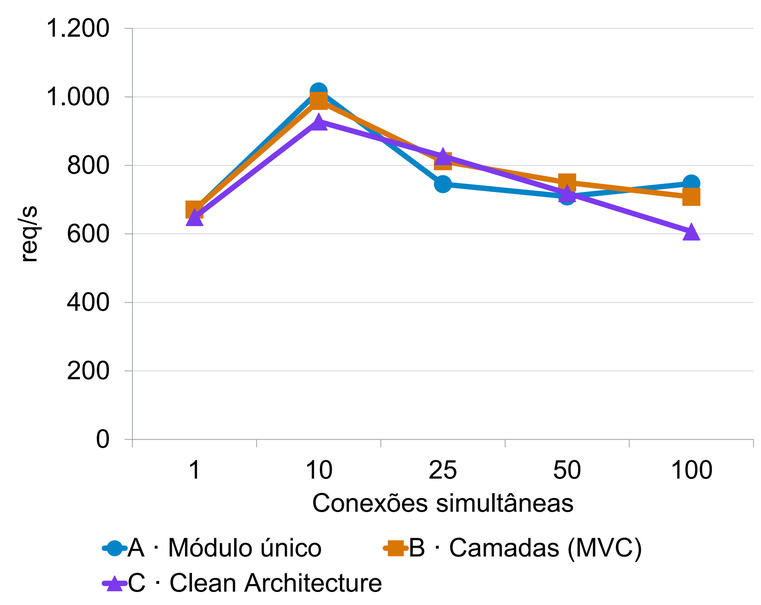
Figura 2 – Throughput por cenário (média de 5 rodadas; 10 conexões; 1 vCPU)



Fonte: elaborado pelos autores (2026).

- **Leitura e misto:** sem diferença detectável ($p \geq 0{,}59$).
- **Criar tarefa:** A 1.060 > B 1.030 > C 1.001 req/s, queda de 5,5% ($p = 0{,}005$, resiste a Bonferroni).

Figura 3 – Throughput sob carga crescente (1 vCPU)



Fonte: elaborado pelos autores (2026).

- **Escalabilidade:** 1 das 8 condições difere (100 conexões, $p = 0{,}042$) e não resiste à correção de Bonferroni (limiar 0,006).
- **Dobrar a CPU (1→2 vCPU):** ×1,15 (A), ×1,31 (B), ×1,24 (C).

5 Indicadores de código

Tabela 1 – Indicadores das três versões

Indicador	A	B	C
Linhas de código	205	356	565
Arquivos	1	16	29
Fan-out médio	0,00	1,63	2,76
Cobertura da regra de negócio	78%	96%	94%
Negócio que importa frameworks	100%	50%	0%

Fonte: elaborado pelos autores (2026). Cobertura dos mesmos 8 testes, sem banco de dados.

6 Discussão

- A **Clean Architecture** isola o domínio (**0%** de dependência de frameworks), mas tem **2,8×** mais código e o maior fan-out (2,76).
- A hipótese de **menor acoplamento** medido por fan-out **não se confirmou**.
- **Testabilidade:** a regra de negócio teve 78% (A), 96% (B) e 94% (C) de cobertura.

7 Conclusões e limitações

- Sem diferença detectável de desempenho na leitura; escrita até **5,5% menor** nas versões mais estruturadas.
- Separar a regra de negócio melhora a testabilidade medida, ao custo de mais código.
- **Limites:** mesmo hospedeiro, CV até 29%, aplicação pequena, uma implementação por estrutura, n = 5; escala horizontal não avaliada.

Referências essenciais

FOWLER, M. **Patterns of enterprise application architecture**. Addison-Wesley, 2002.

MARTIN, R. C. **Clean architecture**. Prentice Hall, 2017.

GARLAN, D.; SHAW, M. An introduction to software architecture. World Scientific, 1993.

BLINOWSKI, G.; OJDOWSKA, A.; PRZYBYŁEK, A. Monolithic vs. microservice architecture. **IEEE Access**, v. 10, p. 20357-20374, 2022.

KRUSKAL, W. H.; WALLIS, W. A. Use of ranks in one-criterion variance analysis. **JASA**, v. 47, 1952.

Agradecimentos

Aos orientadores Prof. Deivison Shindi Takatu e Prof. Glauco Todesco e ao Centro Universitário SENAI São Paulo.

Dados, código e protocolo: github.com/victorgeroto/iniciacao-cientifica-engenharia-de-software